# Introduction aux LLMs

Séance 3 : Entrainement LLMs

---

**ESGI Paris**

Célia Nouri · `celia.nouri@inria.fr`

Semestre 1, 2026–2027

# Agenda

1. Pré-entraînement à grande échelle
2. Lois de Chinchilla
3. Instruction tuning
4. Alignement : RLHF et DPO

# La logique des LLMs modernes

**Étape 1 — Pré-entraînement** : apprentissage du language, tâche non supervisée, immense corpus

**Étape 2 — Fine-tuning / Alignement** : adaptation à une tâche ou à des préférences

```
[Texte brut du web]            [Données annotées / préférences]
        ↓                                ↓
[Pré-entraînement]  →  [Modèle]  →  [Fine-tuning]  →  [Assistant aligné]
   (semaines/mois)     général       (heures/jours)
```

# 1. Pré-entraînement à grande échelle

# Pré-entraînement décodeur : prédire le prochain token

Pour un modèle décodeur (type GPT), la tâche est :

- **Étant donné tous les tokens précédents, prédire le suivant.**

```
Input  : "Le chat est assis sur le"
Cible  : "tapis"
```

- Aucune annotation humaine nécessaire — tout texte est une donnée utilisable
- GPT-3 : ~300 milliards de tokens d'entraînement
- LLaMA 3 : ~15 000 milliards de tokens

# Pré-entraînement encodeur : compléter le blanc "Fill in the blank"

Pour un modèle encodeur (type BERT), la tâche est :

- **Étant donné tous les tokens autours, prédire le token masqué.**

```
Input  : "Le chat est [MASK] sur le tapis"
Cible  : "assis"
```

- Aucune annotation humaine nécessaire — tout texte est une donnée utilisable

# Ce que le modèle apprend implicitement

En apprenant à prédire la suite ou le token masqué, le modèle doit **modéliser** :

- La **grammaire** (pour prédire la bonne forme)
- Les **faits du monde** (Paris est la capitale de la France)
- Le **raisonnement** (si A alors B)
- Le **style** et le **ton**
- Les **relations entre concepts**

Les LLMs modélisent ce qui est **probable** au vue des fréquences d'occurences dans le corpus d'entraînement.

# D'où viennent les données ?

| Corpus | Contenu | Taille approx. |
|---|---|---|
| **Common Crawl** | Archive brute du web, depuis 2008 | plusieurs Po |
| **C4** (Google) | Common Crawl filtré et nettoyé | ~750 Go |
| **The Pile** (EleutherAI, 2020) | 22 sous-corpus : livres, code, PubMed, ArXiv… | 825 Go |
| **FineWeb** (Hugging Face, 2024) | Common Crawl filtré, déduplication poussée | 15 000 Md tokens |
| **Wikipedia, livres, code (GitHub)** | Sources denses en connaissance / structure | variable |

En pratique, un LLM récent mélange **web généraliste + code + livres + données multilingues**, avec des proportions choisies à la main (*data mixture*).
Question de propriété intellectuelle : à qui appartient ces données publiques ?

# Nettoyer les données : pas si simple

Le web brut est plein de **doublons, spam, texte de mauvaise qualité, contenus toxiques**.

**Pipeline typique de nettoyage** :

```
Filtrage de langue → Déduplication → Filtrage qualité (heuristiques + classifieur)
→ Suppression des PII (données personnelles) → Filtrage contenu toxique/NSFW → Mélange des sources
```

**Risques concrets** :

- **Contamination** : si des benchmarks d'évaluation fuitent dans les données d'entraînement, le modèle les "a déjà vus" → scores gonflés (on en parlera plus tard)
- **Droit d'auteur** : procès *New York Times vs OpenAI* (2023) — utiliser des textes protégés pour l'entraînement est-il légal ? Question encore non tranchée juridiquement.
- **Contenus problématiques** : insultes, fausses informations, contenus illégaux...

# 🧩 Quiz — Pré-entraînement

**1.** Pourquoi le pré-entraînement ne nécessite-t-il aucune annotation humaine ?

**2.** Citez un risque concret lié à des données d'entraînement mal nettoyées.

# 🧩 Quiz — Réponses

**1.** La cible (le token suivant) est déjà présente dans le texte lui-même — tout texte brut fournit ses propres paires (input, cible) gratuitement.

**2.** Contenus toxiques (racisme, sexisme) appris par le modèle, contamination des benchmarks (scores gonflés artificiellement) ou violation de droits d'auteur (ex. procès NYT vs OpenAI).

# 2. Lois de Chinchilla

# Une question simple, mais coûteuse

Avec un budget de calcul **fixé** (un nombre de GPU pendant un temps donné), faut-il :

- Entraîner un **très grand modèle** sur **peu de données** ?
- Ou un **modèle plus petit** sur **beaucoup plus de données** ?

Se tromper coûte des millions de dollars de calcul gaspillé.

# Loi de Chinchilla

**"Training Compute-Optimal Large Language Models"**, DeepMind.

**Constat** : la performance suit une loi de puissance prévisible selon la taille du modèle, la quantité de données et le calcul utilisé.

Pour un budget de calcul fixé, **taille du modèle et nombre de tokens doivent croître au même rythme**. La plupart des grands modèles de l'époque (dont Gopher) étaient **trop gros et sous-entraînés**.

**Règle empirique** : environ **20 tokens d'entraînement par paramètre** pour un usage optimal du calcul.

# Une nuance importante : le coût d'inférence

Les lois de Chinchilla optimisent le coût de **l'entraînement**. Mais un modèle déployé à grande échelle est **utilisé (inféré) des milliards de fois** après son entraînement.

**LLaMA (Meta, 2023)** a fait un choix différent : entraîner des modèles **plus petits que l'optimum Chinchilla**, mais sur **encore plus de tokens** que ce que Chinchilla recommanderait — pour réduire durablement le coût d'inférence, quitte à dépenser plus de calcul à l'entraînement.

- "Compute-optimal" (moins cher à entraîner) ≠ "inference-optimal" (moins cher à faire tourner ensuite).

# Conclusions

1. Collection d'un corpus de données massif (web + livres + articles scientifiques...), filtrage qualité.
2. Tokenizer et tokenisation (apprendre un tokenizer à partir d'un corpus d'entraînement spécifique)
3. Déterminser l'architecture du modèle (encoder/décodeur), le nombre de paramètres/ taille du corpus (20 tokens par paramètres)
4. Entraîner le modèle sur le corpus d'entraînement. Non supervisé: prédiction du token suivant ou du token masqué.
   ==> Modèle pré-entraîné

# 3. Instruction tuning

# Le problème du modèle brut

Un modèle juste pré-entraîné sait **continuer du texte**, pas **suivre des instructions**.

```
Prompt : "Explique-moi la photosynthèse."

Modèle pré-entraîné brut :
  "Explique-moi la mitose cellulaire.
   Explique-moi la respiration cellulaire.
   Explique-moi..."          ← continue le style d'une liste de questions

Modèle instruction-tuned :
  "La photosynthèse est le processus par lequel les plantes..."
```

Le modèle de base imite la distribution du **web** — qui contient autant de questions que de réponses.

# Supervised Fine-Tuning (SFT)

**Principe** : ré-entraîner (brièvement) le modèle sur des paires **(instruction, réponse souhaitée)**, rédigées ou validées par des humains.

```
Instruction : "Résume ce texte en 3 phrases : [...]"
Réponse     : "Le texte explique que... En résumé..."
```

- Même fonction de perte que le pré-entraînement (prédiction du prochain token)
- Mais calculée **uniquement sur les tokens de la réponse**, pas sur l'instruction
- Beaucoup moins de données que le pré-entraînement : quelques milliers à quelques millions d'exemples, contre des milliards de documents

# Supervised Fine-Tuning (SFT)

On peut aussi caster des tâches NLP "classiques" (classification de sentiment, de toxicité, NLI, résumé, traduction…) en paires texte → texte, et les mélanger aux instructions.

```
Tâche classique   : classifier("Ce film est nul") → "négatif"
Reformulée en SFT : Instruction "Ce commentaire est-il positif ou négatif ? [...]" → Réponse "négatif"
```

# D'où viennent ces datasets d'instructions ?

| Dataset | Origine | Idée clé |
|---|---|---|
| **FLAN** (Google, 2021-2022) | Reformulation de tâches NLP existantes en instructions | Généralisation à des tâches jamais vues |
| **Self-Instruct** (Wang et al., 2022) | Un LLM génère **lui-même** de nouvelles instructions à partir d'exemples de départ | Réduit le besoin d'annotation humaine |
| **Alpaca** (Stanford, 2023) | 52k instructions générées via Self-Instruct (avec GPT-3), utilisées pour fine-tuner LLaMA | Modèle instruction-tuned pour ~600$ de coût API |
| **Dolly / OpenAssistant** | Instructions rédigées par des humains (employés, volontaires) | Données 100% humaines, ouvertes |

# 🧩 Quiz — Instruction tuning

**1.** Un modèle pré-entraîné brut répond mal à *"Explique-moi la photosynthèse"*. Pourquoi ?

**2.** Sur quels tokens calcule-t-on la perte pendant le SFT : l'instruction, la réponse, ou les deux ?

# 🧩 Quiz — Réponses

**1.** Il n'a appris qu'à **continuer du texte** de la même manière que sur le web ; rien ne le pousse à *répondre* plutôt qu'à *continuer la liste de questions*.

**2.** Uniquement sur les tokens de la **réponse** — l'instruction sert de contexte, pas de cible à prédire.

# 4. Alignement : RLHF et DPO

# Pourquoi ne pas s'arrêter au SFT ?

Même après l'instruction tuning, un modèle peut être :

- **Peu utile** : réponses vagues, trop courtes ou trop longues
- **Non sûr** : contenu dangereux, biaisé, ou toxique
- **Mal calibré** sur les préférences humaines fines (ton, style, niveau de détail)

Objectif d'**alignement** (Anthropic, *HHH*) : un modèle **Helpful, Honest, Harmless**.

- Le SFT apprend "un exemple correct parmi d'autres". L'alignement apprend **une préférence relative** : "cette réponse est meilleure que celle-là."
  Idée générale: il est plus facile d'exprimé une préférence que la "meilleure" réponse à une question.

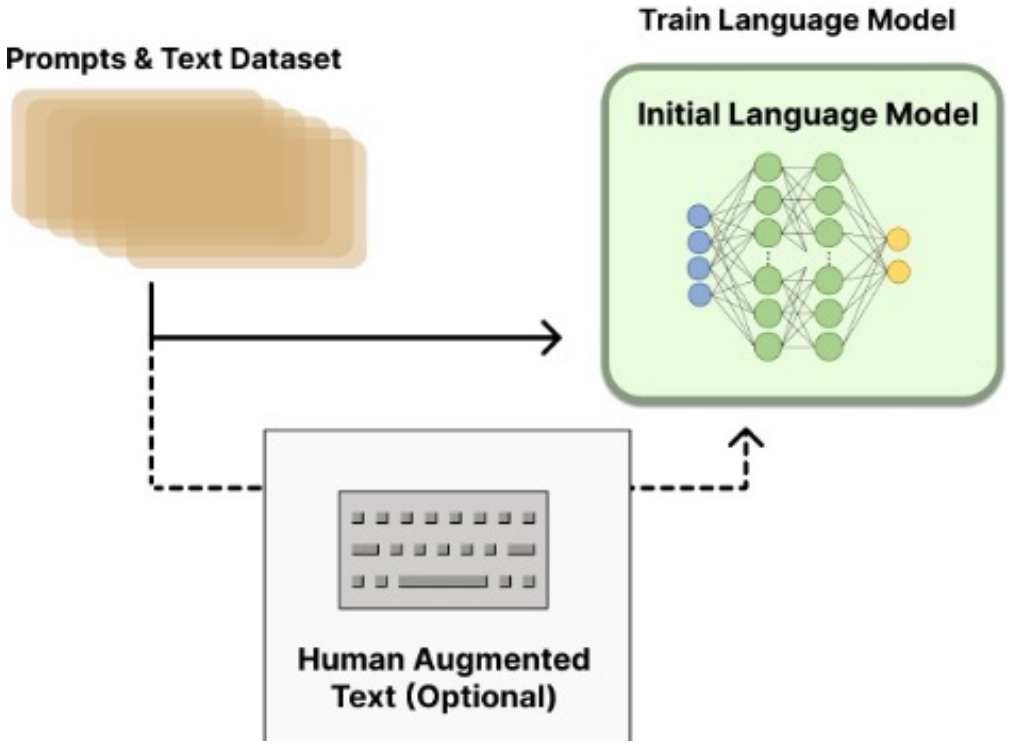
# RLHF — Reinforcement Learning from Human Feedback

**Ouyang et al. (2022, OpenAI)** — pipeline utilisé pour **InstructGPT**, ancêtre direct de ChatGPT.

```
Étape 1 — SFT             : fine-tuning supervisé (déjà vu)
Étape 2 — Reward Model    : entraîner un modèle à noter des réponses
Étape 3 — RL (PPO, proximal policy optimisation)        : optimiser le LLM pour maximiser cette note
```

# Étape 1 : Pre-training et SFT (déjà vus)

1. Pré-entraînez et finetunez votre modèle sur du texte brut, avec un objectif de CLM (*Causal Language Modeling*, prédiction du prochain token).



# Étape 2 : le modèle de récompense

On ne demande **pas** à des humains de noter une réponse dans l'absolu (trop subjectif) — on leur demande de **comparer** deux réponses.

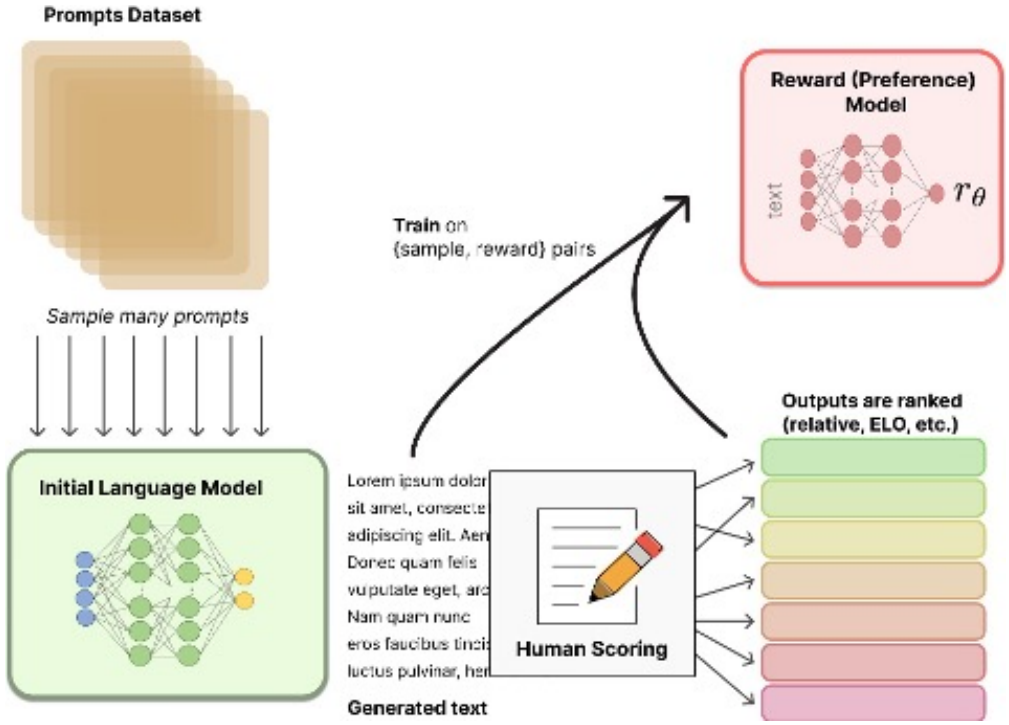
```
Prompt   : "Explique la relativité restreinte."
Réponse A: [claire, correcte]
Réponse B: [confuse]

Humain   : A > B
```

Le **reward model** est entraîné sur des milliers de comparaisons de ce type, pour apprendre à **prédire un score de préférence** pour n'importe quelle réponse.

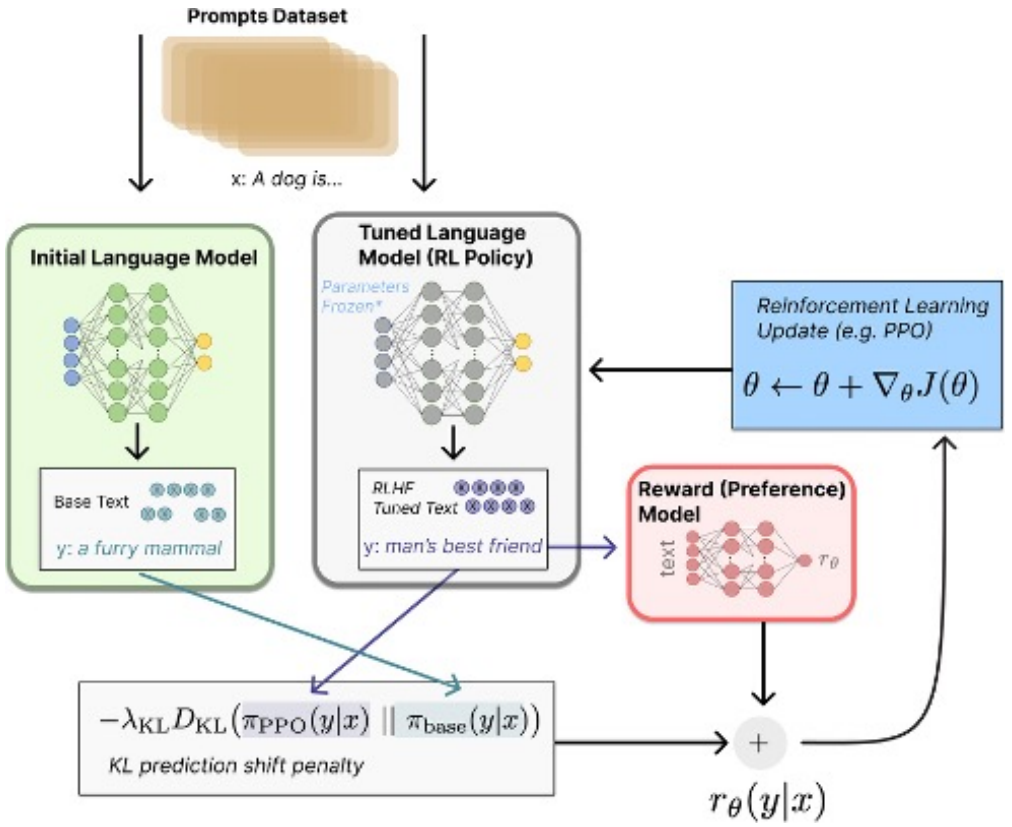
# Étape 2 : le modèle de récompense

2. Entraînez un second modèle de langage à classer les réponses du premier modèle, en se basant sur des préférences humaines.



# Étape 3 : optimiser avec RL (PPO)

Le LLM est ensuite ajusté pour **générer des réponses que le reward model note bien** — via l'algorithme **PPO** (Proximal Policy Optimization).



# Étape 3 : optimiser avec RL (PPO)

```
LLM génère une réponse → Reward Model la note → PPO ajuste les poids du LLM pour augmenter la note future
```

**Garde-fou essentiel** : une pénalité **KL-divergence** empêche le modèle de trop s'éloigner du modèle SFT initial — sinon il apprend à "tricher" avec le reward model (*reward hacking*) au lieu de vraiment s'améliorer.

# Les limites du RLHF

- **Pipeline complexe** : 3 modèles à entraîner et maintenir (SFT, reward model, politique RL)
- **RL instable** : sensible aux hyperparamètres, coûteux à faire converger
- **Reward hacking** : le modèle peut exploiter des failles du reward model plutôt que réellement s'améliorer
- Nécessite de **stocker et faire tourner** le reward model en plus du LLM pendant l'entraînement

→ Motive une simplification : peut-on aligner un modèle **sans** RL ni reward model séparé ?

# DPO — Direct Preference Optimization

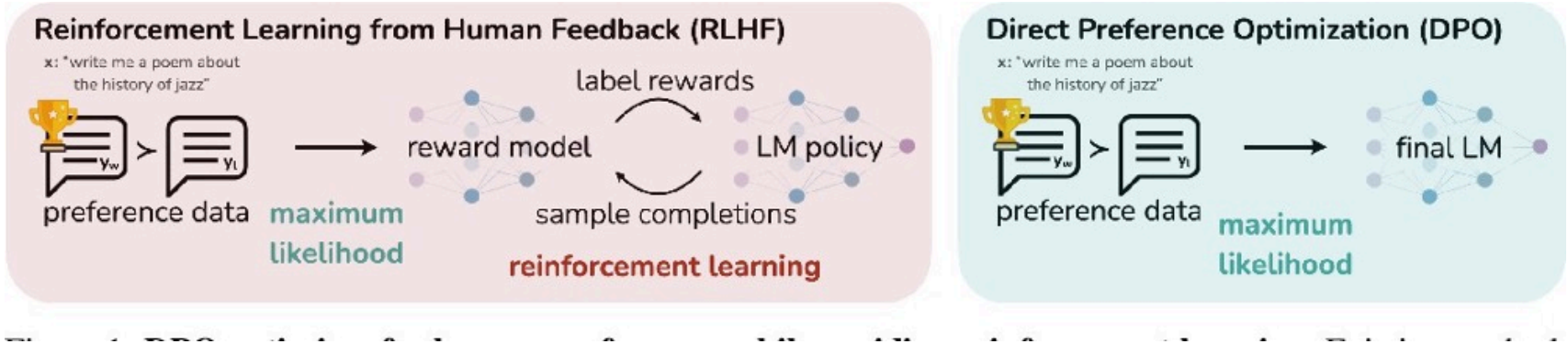
**Rafailov et al. (2023)** — *"Direct Preference Optimization: Your Language Model is Secretly a Reward Model"*

**Idée clé** : on peut montrer mathématiquement qu'un objectif RLHF (reward model + PPO) admet une solution optimale exprimable **directement** en fonction du modèle de langage lui-même.

Résultat : **pas besoin d'entraîner un reward model séparé, ni de faire du RL**. On optimise directement une perte de classification sur les paires (réponse préférée, réponse rejetée).

```
DPO : ajuster le LLM pour augmenter P(réponse préférée)
      et diminuer P(réponse rejetée) — directement, par descente de gradient
```

# RLHF vs DPO


Reinforcement Learning from Human Feedback (RLHF)
x: "write me a poem about the history of jazz"
preference data
maximum likelihood
reward model
label rewards
LM policy
sample completions
reinforcement learning
Direct Preference Optimization (DPO)
x: "write me a poem about the history of jazz"
preference data
maximum likelihood
final LM

# RLHF vs DPO

| | RLHF | DPO |
|---|---|---|
| **Modèles à entraîner** | SFT + Reward Model + Policy RL | SFT + un seul modèle final |
| **Algorithme** | PPO (reinforcement learning) | Descente de gradient supervisée |
| **Stabilité** | Sensible, coûteux à régler | Plus simple, plus stable |
| **Popularité (2023+)** | Standard historique (InstructGPT) | Largement adopté (Zephyr, Mixtral-Instruct, LLaMA 3…) |

DPO ne remplace pas la **collecte de préférences humaines** — seulement la façon d'optimiser le modèle à partir de ces préférences.

# 🧩 Quiz — RLHF & DPO

**1.** Pourquoi demande-t-on aux humains de **comparer** deux réponses plutôt que de noter une réponse seule ?

**2.** À quoi sert la pénalité KL-divergence pendant l'étape RL du RLHF ?

**3.** Quelle est la principale simplification qu'apporte DPO par rapport au RLHF classique ?

# 🧩 Quiz — Réponses

**1.** Comparer deux réponses est **beaucoup plus fiable et cohérent** entre humains que noter une réponse dans l'absolu — un jugement relatif est plus facile à faire qu'un jugement absolu.

**2.** Elle empêche le modèle de trop s'éloigner de sa version SFT initiale, ce qui évite le **reward hacking** (exploiter des failles du reward model).

**3.** DPO élimine le besoin d'un **reward model séparé** et d'un algorithme de **RL (PPO)** : l'alignement se fait par une seule perte supervisée, directement sur les paires de préférences.

# Récapitulatif : comment construit-on un LLM ?

```
Texte brut du web (nettoyé)
    ↓  pré-entraînement à grande échelle (loi de Chinchilla : taille ↔ données)
Modèle de base (complète du texte)
    ↓  instruction tuning (SFT)
Modèle qui suit des instructions
    ↓  alignement (RLHF ou DPO)
Assistant aligné sur les préférences humaines
    ↓  évaluation (benchmarks + humains + LLM-as-judge)
Modèle déployé
```

# Ressources

📄 **Scaling Laws** : Kaplan et al. (2020); arxiv.org/abs/2001.08361
📄 **Chinchilla** : Hoffmann et al. (2022); arxiv.org/abs/2203.15556
📄 **InstructGPT (RLHF)** : Ouyang et al. (2022); arxiv.org/abs/2203.02155
📄 **Self-Instruct** : Wang et al. (2022); arxiv.org/abs/2212.10560
📄 **DPO** : Rafailov et al. (2023); arxiv.org/abs/2305.18290

🛠️ **Hugging Face TRL** (SFT, DPO, PPO) : huggingface.co/docs/trl

# Lab : Aujourd'hui (1h)

**Partie 1** : Explorer un dataset d'instructions (Alpaca / Dolly)

Observer le format (instruction, réponse), identifier des exemples ambigus ou de mauvaise qualité.

**Partie 2** : SFT léger sur un petit modèle avec `TRL` / `transformers`